THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

Application Layer – HTTP and HTML

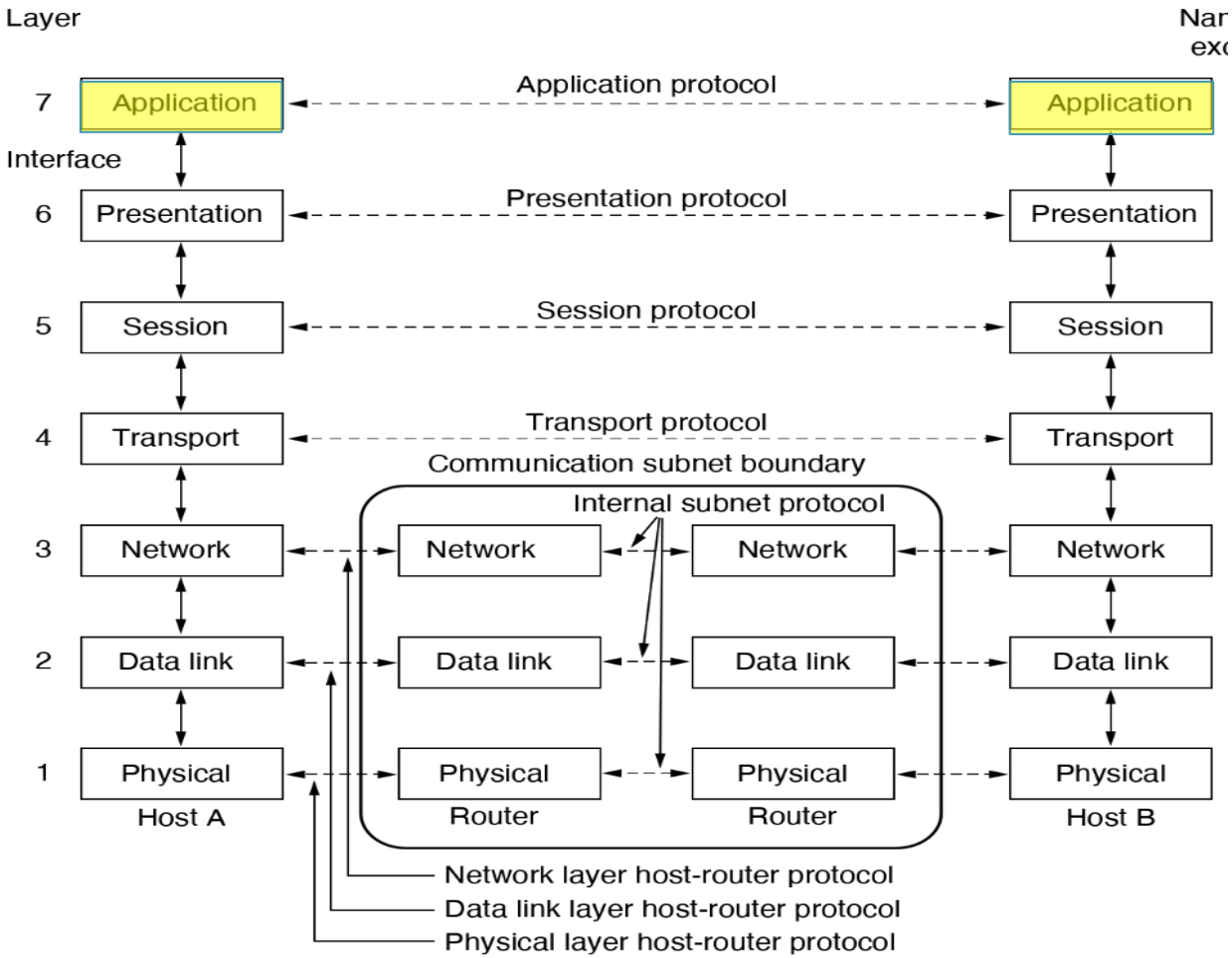
Dr Lachlan Andrew

Recap

Email

- SMTP – main email protocol, push-based
- POP/IMAP – pull-based for collection by the final recipient
- Very “chatty” protocols, with many round-trip exchanges

In the stack: Application layer



- Use data
- Tidy up e2e
- Get data e2e
- Tidy up p2p
- Get data p2p

HTTP

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

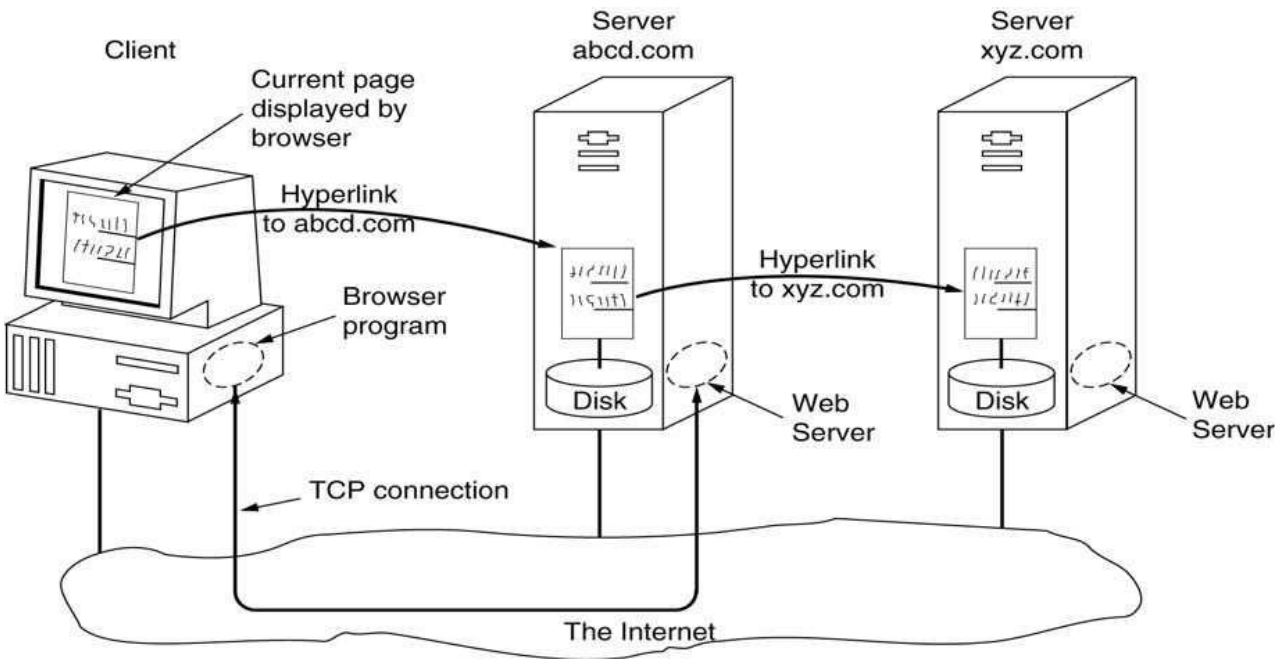
```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

© 2025 University of Melbourne

Summary

- Application Layer
  - HTTP (the web protocol), and in relation to it, HTML

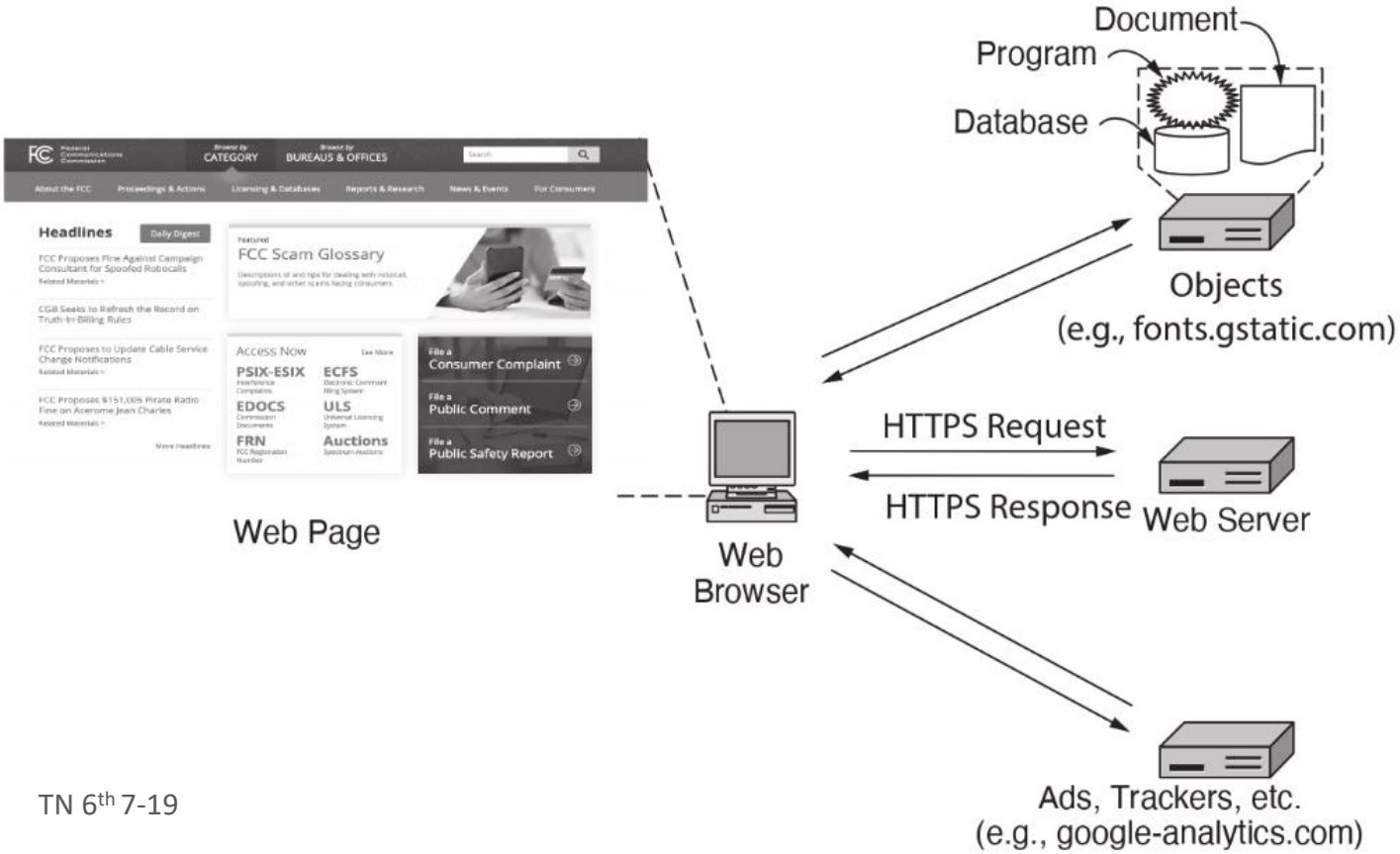
WWW - Components



TN 4th 7-19

- Client - typically a browser based access to pages
- Server - daemon based content delivery of pages
- URL ≈ Protocol + DNS Name + file name

WWW - Architecture



TN 6th 7-19

HTTP - Overview

- HyperText Transfer Protocol
  - Defined everything needed for the web
- Resources are referred to by URLs

URL/URI

- Uniform Resource Locator
  - Defined in original HTTP specification
  - An address for a resource
  - Can be absolute “http://www.google.com” or relative “./nextpage.html”
- Separate specification by W3C in 1998 for URI
  - Uniform Resource Identifier, includes URLs, but “names” like ISBNs
- scheme:[//[user[:password]@]host[:port]][/path][?query][#fragment]
- `abc://user:passwd@example.com:123/path/data?key=value#frag1`

HTTP – Protocol Overview

- Overview:
  - Client initiates TCP connection (creates socket) to server, port 80
    - (Can use TLS or QUIC instead of TCP, be we'll keep saying "TCP")
  - Server accepts TCP connection from client
  - HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
  - TCP connection closed
- Connections:
  - HTTP 1.0 – single-use connection
  - HTTP 1.1 – persistent connections, additional headers

Non-persistent HTTP

Suppose the user enters http://www.somesite.com.au/index.html
(the page contains text and references to 10 images)

1a. HTTP Client initiates TCP connection to HTTP server (process) at www.somesite.com.au on port 80

1b. HTTP server at host www.somesite.com.au waiting for TCP connection at port 80. "accepts" connection, notifying client

2. HTTP client sends HTTP request message (containing URL) into TCP connction socket. Message indicates that client wants object index.html

3. HTTP server receives request message, forms response message containing requested object, and sends message into its socket

4. HTTP server closes TCP connection

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 refrenced jpeg objects

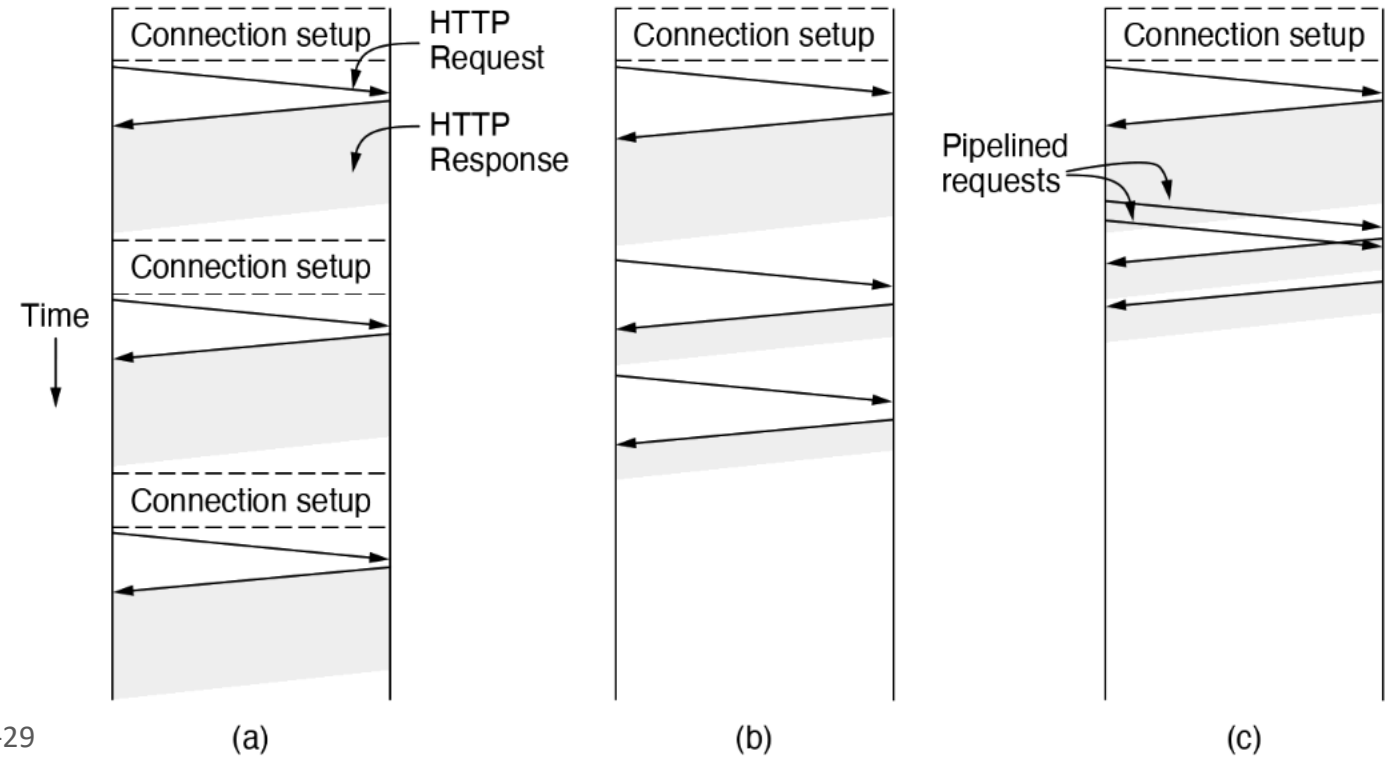
6. Steps 1-5 repeated for each of 10 jpeg objects

© 2025 University of Melbourne

Persistent vs. Non-persistent

- Non-persistent:
  - requires 2 “response times” (one to initiate TCP connection and one for initial HTTP request) per object + file transmission time
  - OS overhead for each TCP connection
  - browsers often open parallel TCP connections to fetch referenced objects
- Persistent:
  - server leaves connection open after sending response
  - subsequent HTTP messages between same client/server sent over open connection
  - client sends requests as soon as it encounters a referenced object, reducing overall response time

HTTP Request Connection



TN 6th 7-29

- HTTP with (a) multiple connections and sequential requests.
  (b) A persistent connection and sequential requests.
  (c) A persistent connection and pipelined requests.

HTTP – Summary of key steps

- Steps that occur when a link is selected:
  - Browser determines the URL
  - Browser asks DNS for the IP address of the server (Resolving URL)
  - DNS replies
  - The browser makes a TCP connection
  - Sends HTTP request for the page
  - Server sends the page as HTTP response
  - Browser fetches other URLs as needed, possibly opening more TCP
  - The browser displays the page (progressively, as content arrives)
  - The TCP connections are released

HTTP – Request Methods

HTTP Method	Safe	Idempotent	Cacheable
GET	Yes	Yes	Yes
HEAD	Yes	Yes	Yes
POST	No	No	Yes/No
PUT	No	Yes	No
DELETE	No	Yes	No
CONNECT	No	No	No
OPTIONS	Yes	Yes	No
TRACE	Yes	Yes	No
PATCH	No	No	No

- Idempotent – multiple identical requests have same effect
- Safe – Only for information retrieval, should not change state

Wireshark Example

No.	Time	Source	Destination	Protocol	Length	Info
11	10.324525144	10.0.2.15	184.87.112.241	TCP	74	47536 → 80 [SYN] Seq=0 Win=64240 Le
12	10.331723283	184.87.112.241	10.0.2.15	TCP	60	80 → 47536 [SYN, ACK] Seq=0 Ack=1 W
13	10.331762371	10.0.2.15	184.87.112.241	TCP	54	47536 → 80 [ACK] Seq=1 Ack=1 Win=64
14	10.522991961	10.0.2.15	184.87.112.241	HTTP	2177	GET /news/2020-02-27/who-are--scier
15	10.524109383	184.87.112.241	10.0.2.15	TCP	60	80 → 47536 [ACK] Seq=1 Ack=1461 Wir
16	10.524133844	184.87.112.241	10.0.2.15	TCP	60	80 → 47536 [ACK] Seq=1 Ack=2124 Wir
17	10.533234027	184.87.112.241	10.0.2.15	HTTP	536	HTTP/1.1 302 Moved Temporarily
18	10.533256270	10.0.2.15	184.87.112.241	TCP	54	47536 → 80 [ACK] Seq=2124 Ack=483 W

```
▶ Frame 14: 2177 bytes on wire (17416 bits), 2177 bytes captured (17416 bits) on interface 0
▶ Ethernet II, Src: PcsCompu_80:ae:e9 (08:00:27:80:ae:e9), Dst: RealtekU_12:35:02 (52:54:00:12:35:02)
▶ Internet Protocol Version 4, Src: 10.0.2.15, Dst: 184.87.112.241
▶ Transmission Control Protocol, Src Port: 47536, Dst Port: 80, Seq: 1, Ack: 1, Len: 2123
▼ Hypertext Transfer Protocol
  ▶ GET /news/2020-02-27/who-are--scientists-professionals-who-say-no-climate-emergency/11734966 HTTP/1.1\r\n
    Host: www.abc.net.au\r\n
    User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:73.0) Gecko/20100101 Firefox/73.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\n
    Accept-Language: en-AU,en-US;q=0.7,en;q=0.3\r\n
    Accept-Encoding: gzip, deflate\r\n
    Connection: keep-alive\r\n
  ▶  [truncated]Cookie: ABC_News_Visits=1564619576732%3A1582683283093%3A134; ABC_News_Cookies=134; ABC_News_Msg=%
    Upgrade-Insecure-Requests: 1\r\n
    \r\n
    [Full request URI: http://www.abc.net.au/news/2020-02-27/who-are--scientists-professionals-who-say-no-climate
    [HTTP request 1/1]
    [Response in frame: 17]
```

HTTP Request Example

request line
(GET, POST, HEAD)

header lines

Blank line
(2 CR/LF)
indicates end of
message

```
GET /somedir/page.html HTTP/1.1
Host: www.somesite.com.au
User-agent: Mozilla/4.0
Connection: close
Accept-language: fr

(extra new line)
```

HTTP Response Codes

Code	Meaning	Examples
1xx	Information	100 – server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 try again later

HTTP Response Example

Status line (protocol status code and phrase)

header lines

Data, e.g., requested HTML file

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 06 Aug 2009 12:00:15 GMT
Server: Apache/2.2.11 (Unix)
Last-modified: Mon, 22 Jun 2009
Content-Length: 6821
Content-Type: text/html

<html><head>…
```

Comparison with SMTP

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

CLIENT:

From: alice@crepes.fr

To: bob@hamburger.edu

Do you like ketchup?

How about pickles?

SERVER:

250 Message accepted

HTTP Headers

Don't learn all these.
Just remember/understand those in the examples.

Header	Type	Description
User-Agent	Request	Information about the browser and its platform
Accept	Request	The type of pages the client can handle
Accept-Charset	Request	The character sets that are acceptable to the client
Accept-Encoding	Request	The compression formats the client can handle
Accept-Language	Request	The natural languages the client can handle
If-Modified-Since	Request	Time and data to check freshness
If-None-Match	Request	Previously sent tags to check freshness
Host	Request	The server's DNS name
Authorization	Request	A list of the client's credentials
Referer	Request	The previous URL from which the request came
Cookie	Request	Previously set cookie sent back to the server
Set-Cookie	Response	Cookie for the client to store
Server	Response	Information about the server

© 2025 University of Melbourne

Header	Type	Description
Content-Encoding	Response	How the content is encoded (e.g., gzip)
Content-Language	Response	The natural language used in the page
Content-Length	Response	The page's length in bytes
Content-Type	Response	The page's MIME type
Content-Range	Response	Identifies a portion of the page's content
Last-Modified	Response	Time and date the page was last changed
Expires	Response	Time and date when the pages stops being valid
Location	Response	Tells the client where to send its request
Accept-Ranges	Response	Indicates the server will accept byte range requests
Date	Both	Date and time the message was sent
Range	Both	Identifies a portion of a page
Cache-Control	Both	Directives for how to treat cache
Etag	Both	Tag for the contents of the page
Upgrade	Both	The protocol the sender wants to switch to

HTTPS –HTTP over TLS

- Not really a separate protocol – mostly just ordinary HTTP over a different service
  - Runs over TLS – transport layer security – instead of directly over the transport layer
- Uses port 443 instead of port 80
- Everything is encrypted (e.g., URLs), not just the payloads

© 2025 University of Melbourne

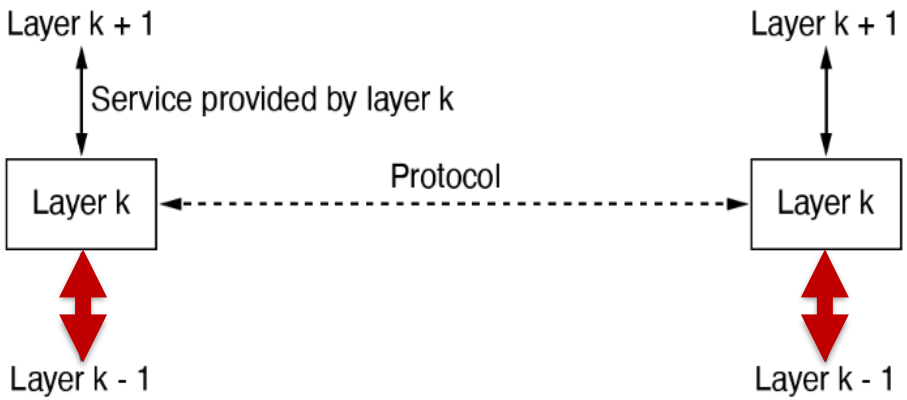
HTTP/2

- Compatible at a high level with HTTP 1.1
- Aim: decrease latency
  - Compress HTTP headers
  - Server push
    - if I've just send HTML containing an <img> tag, I'll start sending the image before you ask for it
  - Multiplexing requests over a TCP connection
    - one image doesn't have to wait until the previous is loaded
- Negotiation
  - Client and server can choose HTTP/1.1 or HTTP/2 or something else

© 2025 University of Melbourne

HTTP/3

- Aim: decrease latency
  - Runs over QUIC not TCP: more parallelism, esp. with packet loss
- Application layer protocol basically unchanged from HTTP/2
- Proposed standard



And finally…

- Tracking with cookies is well known
- Tracking companies have expanded beyond simple cookies
  - Plug-in, browser fingerprinting
- https://coveryourtracks.eff.org/ Project to research tracking techniques in browsers
- How unique is your browser by Peter Eckersley (EFF) : https://coveryourtracks.eff.org/static/browser-uniqueness.pdf

Acknowledgement

- The slides were based on slides prepared by Chris Culnane, based on material developed previously by: Michael Kirley, Zoltan Somogyi, Rao Kotagiri, James Bailey and Chris Leckie.
- Some of the images included in the notes were supplied as part of the teaching resources accompanying the text books listed on the previous slides.
  - (And also) Computer Networks, 6th Edition, Tanenbaum A., Wetherall. D. https://ebookcentral.proquest.com/lib/unimelb/detail.action?docID=6481879
- Textbook Reference: Section 2.1 and 2.2, and related topics from pp.199-210

© 2025 University of Melbourne

Background

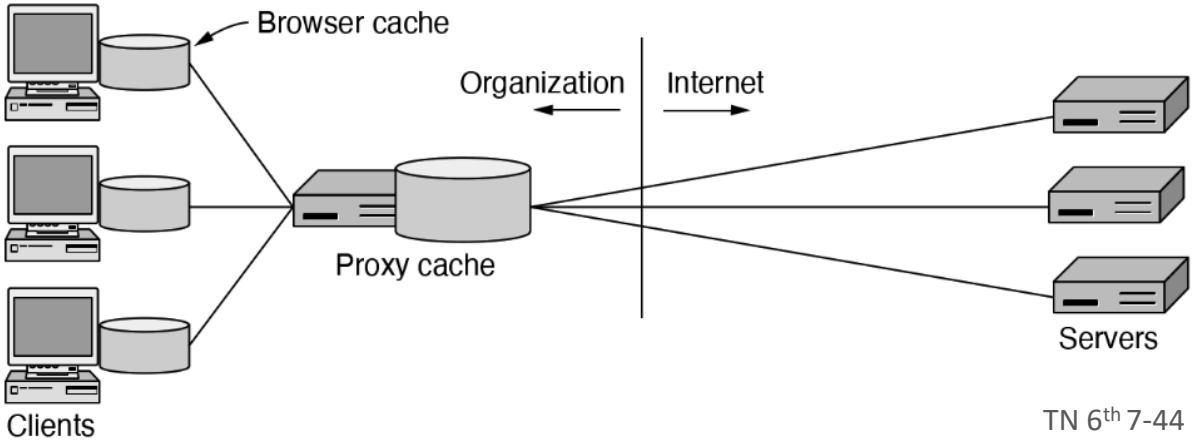
The remainder of these slides are not assessable

They are just for the curious

© 2025 University of Melbourne

Web proxy

- Used for caching, security and IP address sharing
- The browser sends all HTTP requests to the proxy. The proxy returns objects in its cache or else the proxy requests object from origin server, then returns object to client.
- Note: the proxy server acts as both client and server.



TN 6th 7-44

Cookies

- The network stores no state about web sessions
- Cookies can place small amount (<4Kb) of information on the users computer and re-use deterministically (RFC 2109)
- Cookies have 5 fields
  - domain, path, content, expiry, security
- How to keep state – maintain state at sender/receiver over multiple transactions; http messages carry “state”
- Questionable mechanism for tracking users (invisibly perhaps) and learning about user behaviour
  - e.g., competitor snooping, undesirable content etc.

Example Cookies

amazon.com.au

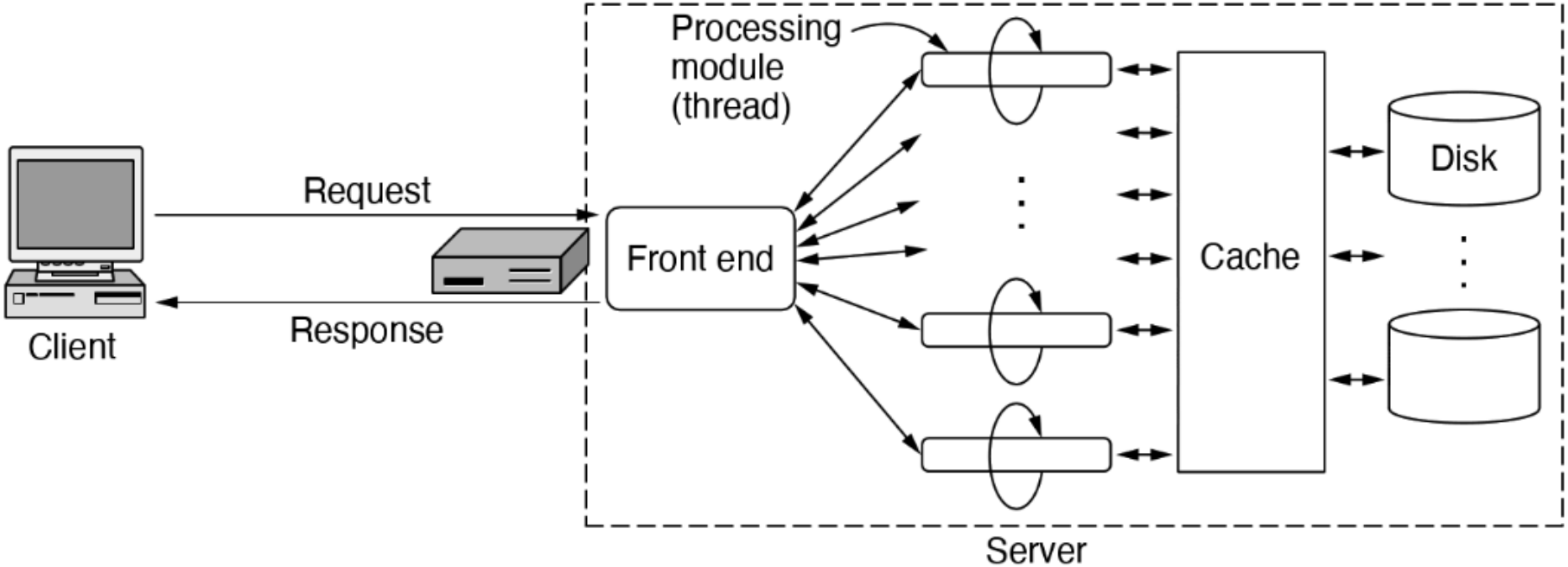
Name	Value	Domain	Expires	HTTPOnly	Secure
Session-id	356-7554479-6471342	.amazon.com.au	2036-01-01.		
Session-id-time	2082787201l	.amazon.com.au	2036-01-01.		
ad-id	A3kfU1c7DE3Wqz474A25Zfs	.amazon.adsystem.com	2037-01-01.		

nytimes.com

Name	Value	Domain	Expires	HTTPOnly	Secure
ad-id	A3kfU1c7DE3Wqz474A25Zfs	.amazon.adsystem.com	2037-01-01.		

Multi-threaded Web Server

TN 6th 7-22

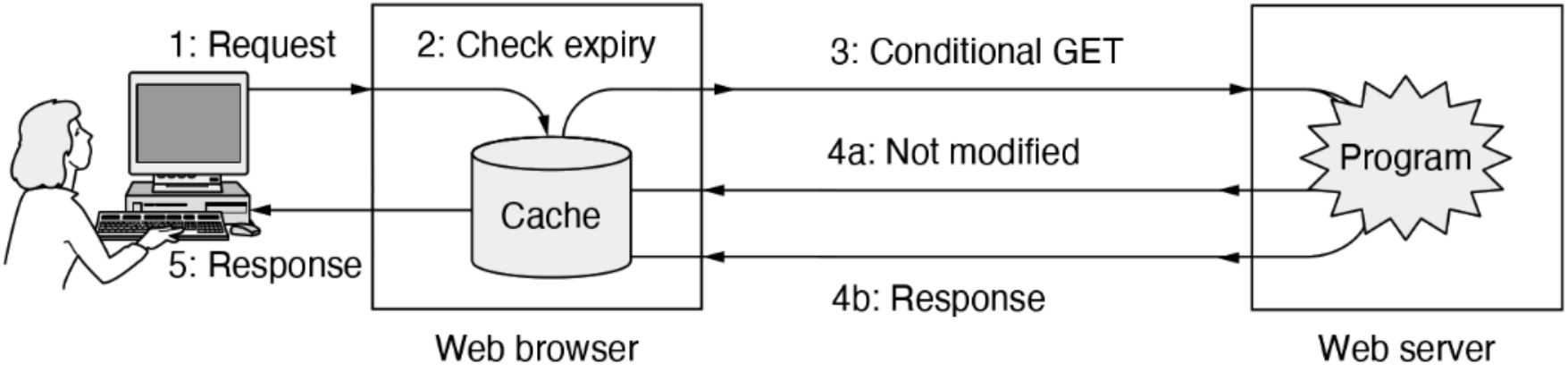


- A multithreaded Web server with a front end and processing modules.

Multi-threaded Web Server – dynamic

- A processing module performs a series of steps:
  - Resolve name of Web page requested.
  - Perform access control on the Web page.
  - Check the cache.
  - Fetch requested page from disk or run program
  - Determine the rest of the response
  - Return the response to the client.
  - Make an entry in the server log.

Web Cache



TN 6th 7-28

Static web documents

- HTML - Hypertext Markup Language
  - a simple language designed to encode both content and presentational information
  - Plain text encoding, with browser based rendering
  - Restricted to ISO-8859 Latin-1 character set (internationalisation not introduced until XHTML with UTF encodings)
- Web Page Components
  - Structural divisions:
    - Head <head> ... </head>
    - Body <body> ... </body>
  - Syntactically Restricted Tag Sets
  - Attributes & Values <img src="pic1.gif" alt="pic1.gif">

Item	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
Hyperlinks	x	x	x	x	x
Images	x	x	x	x	x
Lists	x	x	x	x	x
Active maps & images		x	x	x	x
Forms		x	x	x	x
Equations			x	x	x
Toolbars			x	x	x
Tables			x	x	x
Accessibility features				x	x
Object embedding				x	x
Style sheets				x	x
Scripting				x	x
Video and audio					x
Inline vector graphics					x
XML representation					x
Background threads					x
Browser storage					x
Drawing canvas					x

Background

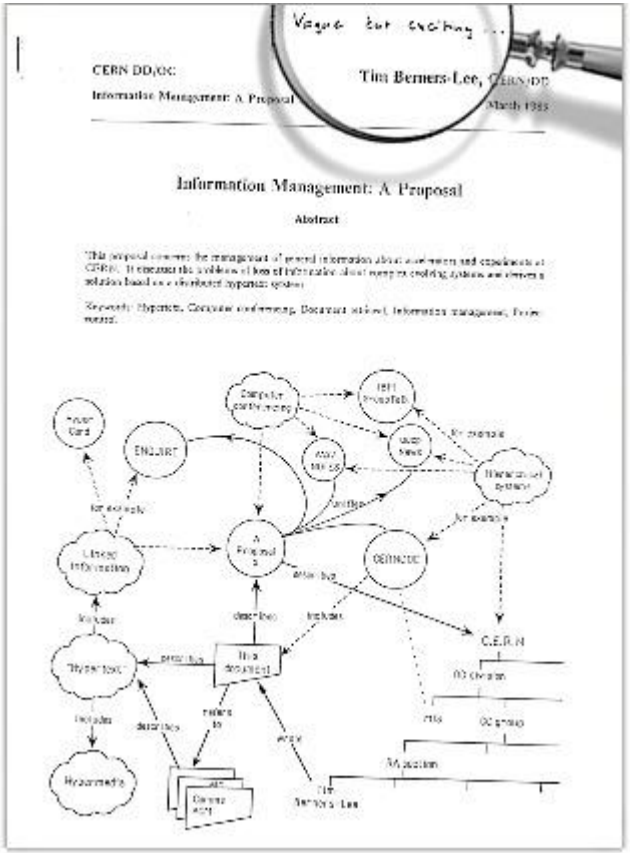
The remainder of these slides are not assessable

They are just for the curious

© 2025 University of Melbourne

World Wide Web – A Short History

- Sir Tim Berners-Lee
  - Saw many online databases with different access mechanisms (FTP, Gopher, …)
  - 1989 wrote the proposal “a large hypertext database with typed links” (No takers)
  - by 1990, had designed and built: HTTP, HTML, httpd, WorldWideWeb (browser)
  - 1992 left for MIT, after CERN IT Head described it as a misallocation of resources
- Hypertext
  - Ted Nelson coined the term in 1963
  - Creation and use of linked content



World Wide Web – A Short History

- The vision was that HTTP would be the “glue” between data on different existing protocols
  - e.g., FTP (file transfer protocol) – many files available for download
- GOPHER – distributed database developed at U. Minnesota in 1991
  - Hierarchical file structure
  - More suited for text interfaces – lower network overhead
  - February 1993, charging for server
- May 1994 first International WWW Conference (at CERN)
- September 1994 W3C formed (DARPA & European Community)
  - Standardisation of web technologies – royalty free
- Browser wars 1994-1998 (Microsoft vs. Netscape)
- 1999 – 2001 .com boom
- 2002+ Ubiquitous web
- Web 2.0 – semantic web, social media

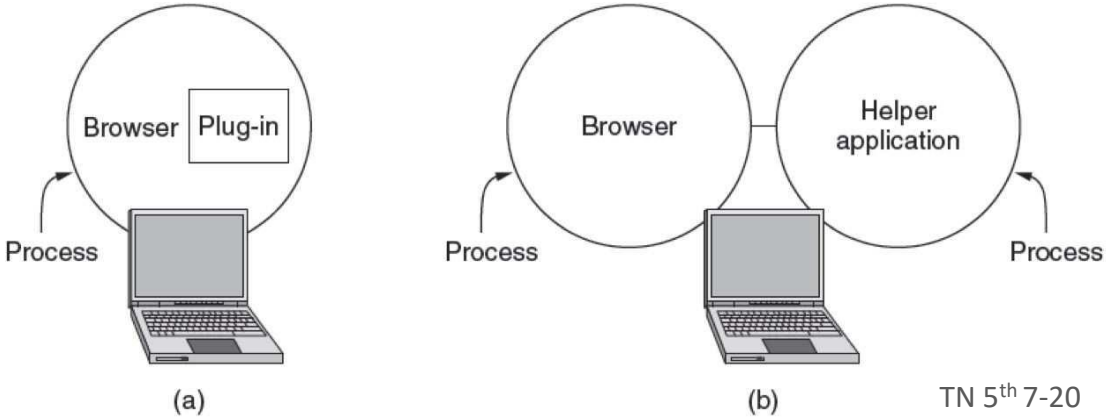
Beyond HTML

- HTML was originally an instance of SGML
  - standard generalized markup language
- People wanted an HTML-like language to describe data that is not hypertext – but SGML is too general / “heavy”

- XML (Extensible Markup Language)
  & XSL (Extensible Stylesheet Language)
  - Primary feature: separation of content and presentational markup
  - Stringent validation requirements
- XHTML
  - Essentially an expression of HTML 4.0 as valid XML
  - Major differences to HTML 4.0 are the requirements for conformance, case folding, well-formedness, attribute specification, nesting and embedding, and inclusion of a document type identifier

Client side processing

- Plugins/Extensions - integrated software module which executes inside the browser,
  - direct access to online context
- Helper - separate program which can be instantiated by the browser, but can only access local cache of file content
  - application/pdf
  - application/msword

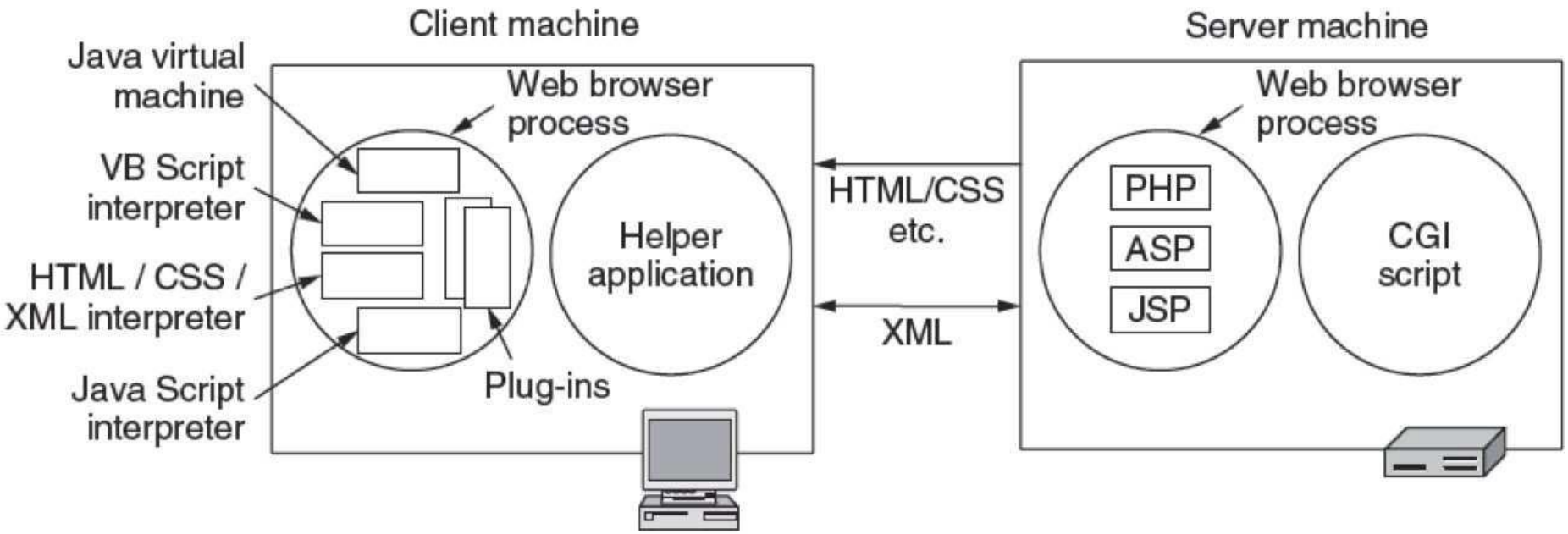


TN 5th 7-20

Server side processing – static page

- 5 step process:
  - Accept TCP Connection from client (browser)
  - Identify the file requested
  - Get the specified file from the local storage (disk, RAM, …)
  - Send the file to the client
  - Release the TCP connection

Dynamic Content



TN 5th 7-35

Dynamic Content

```
<html>
<body>
<form action="action.php" method="post">
<p> Please enter your name: <input type="text" name="name"> </p>
<p> Please enter your age: <input type="text" name="age"> </p>
<input type="submit">
</form>
</body>
</html>
```

(a)

```
<html>
<body>
<h1> Reply: </h1>
Hello <?php echo $name; ?>.
Prediction: next year you will be <?php echo $age + 1; ?>
</body>
</html>
```

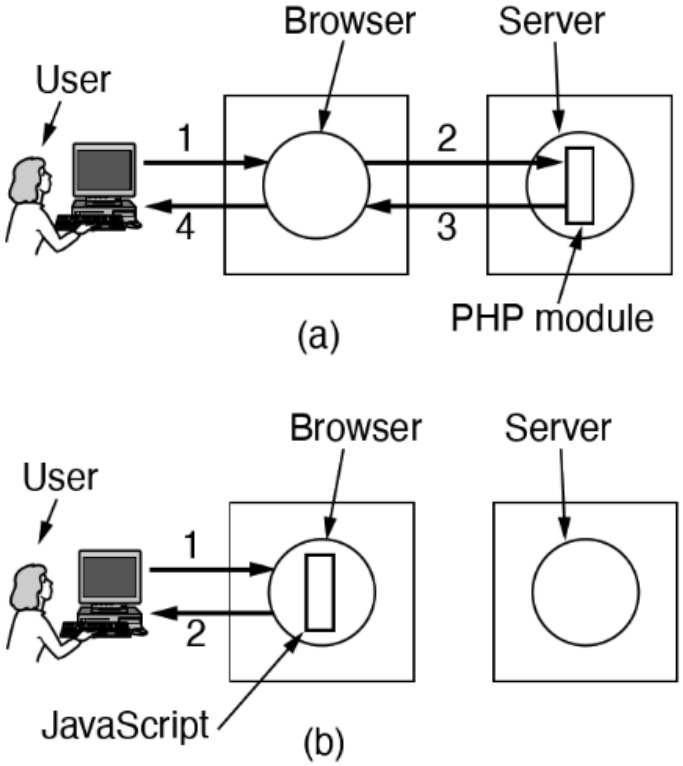
(b)

```
<html>
<body>
<h1> Reply: </h1>
Hello Barbara.
Prediction: next year you will be 33
</body>
</html>
```

(c)

From TN 5th Edition, Figure 7-30

Scripting



TN 6th 7-24

Client-side Scripting

- Technologies for producing interactive web applications include:
- JavaScript
- Java Applets - compiled Java code (platform independent)
- ActiveX - compiled code for Windows
- AJAX
  - HTML and CSS: present information as pages.
  - DOM: change parts of pages while they are viewed.
  - XML: let programs exchange data with the server.
  - An asynchronous way to send and retrieve XML data.
  - JavaScript as a language to bind all this together